

# Latent Semantic Indexing (LSI) Assignment

Dilara Alkanalka 50251145

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1. Introduction .....</b>                    | <b>2</b> |
| <b>2. Methodology &amp; Implementation.....</b> | <b>2</b> |
| <b>3. Experiments and Results .....</b>         | <b>3</b> |
| <b>4. Discussion and Analysis .....</b>         | <b>7</b> |
| <b>5. Conclusion .....</b>                      | <b>7</b> |
| <b>6. References .....</b>                      | <b>8</b> |

## 1. Introduction

The aim of Information Retrieval Systems is to catch documents that are relevant to user's given query from a dataset. In many traditional Information Retrieval techniques, this process is done by using exact query matching, however this approach may fail to retrieve the most relevant documents or sources in a real-life scenario. In this project, Latent Semantic Indexing (LSI) is used to enhance the IR system. LSI lets the system catch hidden semantic structures during the document scanning, leading to an improved retrieval quality by recognizing relationships between the words and documents in the dataset. This way the IR system can retrieve documents that are semantically related, even if that document does not have the exact match of the terms in the query.

In this project, the IR system is tested with and without the LSI and the performance is compared to evaluate the retrieval effectiveness.

## 2. Methodology & Implementation

In this project, a basic Information Retrieval (IR) system was enhanced by implementing Latent Semantic Indexing (LSI). Firstly documents were preprocessed by tokenizing and lowercasing the text using a 'preprocess' function. In the initial tests stopwords were kept, however for a more accurate experimenting with the current size of the dataset stopwords were removed on the later tests to improve retrieval efficiency. Next, a Term Document Matrix was created using CountVectorizer, representing each document by its term frequency, which corresponds to 'A matrix' in the theoretical LSI formula. Then, Singular Value Decomposition (SVD) was applied using 'TruncatedSVD' to reduce the dimensionality of the term-document matrix, which corresponds to  $A=U\Sigma V^T$  decomposition step, retaining the top k singular values and vectors to capture major semantic structure of the dataset. Following decomposition, both the documents and the user query were projected into the lower-dimensional semantic space, which was done by transforming the query and documents using the fitted SVD model ( $X_{lsi}$  and  $query\_vec\_lsi$ ). Finally, cosine similarity was calculated between the query vector and each document vector in the space and documents were ranked in decreasing order of similarity, allowing retrieval of the most contextually relevant results. So this system fully implements the core ideas of LSI: constructing a term-document matrix, performing SVD, projecting into semantic space, and ranking based on latent concept similarity.

### 3. Experiments and Results

A collection of news articles related to politics which vary in length and style were collected in a plain text format as a dataset for this project. The articles cover topics such as international conflicts, elections, and government policies. Queries were taken as user input during runtime, and results were generated as comparison of different IR models.

First, to evaluate retrieval performance three models were tested and compared: Boolean retrieval using strict term matching, cosine similarity based on vector space representation, and an enhanced model using Latent Semantic Indexing (LSI) to capture latent semantic relationships.

Entered query: **Germany defense spending**

```
=====
BOOLEAN RETRIEVAL (AND logic):
artc93.txt
artc105.txt
=====
TOP 10 DOCUMENTS WITHOUT LSI (Cosine Similarity):
artc91.txt (score: 0.1638)
artc70.txt (score: 0.1622)
artc74.txt (score: 0.1162)
artc93.txt (score: 0.1139)
artc79.txt (score: 0.1107)
artc83.txt (score: 0.1046)
artc80.txt (score: 0.0737)
artc82.txt (score: 0.0557)
artc104.txt (score: 0.0550)
artc105.txt (score: 0.0543)
=====
TOP 10 DOCUMENTS WITH LSI:
artc91.txt (score: 0.3140)
artc70.txt (score: 0.3099)
artc74.txt (score: 0.2233)
artc93.txt (score: 0.2179)
artc79.txt (score: 0.2130)
artc83.txt (score: 0.1996)
artc80.txt (score: 0.1414)
artc82.txt (score: 0.1063)
artc104.txt (score: 0.1053)
artc105.txt (score: 0.1052)
```

For this query, the Boolean retrieval model returned two documents: artc93.txt and artc105.txt that contained all three query terms explicitly. (Title of artc93.txt is: “German Parlieament Passes Historic Debt Reform, Paving the Way for Higher Defense Spending” and title of artc105.txt is: “Europe Stocks Lower; UK Borrowing Costs Fall After Spring Fiscal Statement and Gilt Issuance Announcement”) When using cosine similarity without LSI, these documents were included in the top 10 results, but they were not ranked highest. Documents such as artc91.txt and artc70.txt, which may contain fewer or different combinations of query terms, received higher scores due to greater overall vector similarity with the query. (Title of artc91.txt is: “European Leaders Push for Even More Defense Spending-Despite Plans for \$867 Billion ‘Rearm’ Package) After applying LSI, these same documents (artc91.txt, artc70.txt) achieved even higher scores, demonstrating the effectiveness of LSI in identifying semantically related documents. Although artc105.txt was retrieved by the Boolean model due to containing all the query terms, artc91.txt ranked significantly higher in cosine similarity (with and without LSI), as it was more relevant to the user’s search intent which highlights the importance of contextual relevance. Furthermore, with LSI applied, the similarity score of artc91.txt increased even more, enhancing confidence in its semantic alignment with the query. This shows how LSI strengthens retrieval by uncovering deeper relationships between documents and queries, improving both accuracy and user satisfaction.

For the remaining examples, it was decided to focus on the comparison between cosine similarity with and without LSI, as Boolean retrieval was previously shown to be too restrictive and less effective in capturing relevance and user intent.

Next query: **Russia Ukraine negotiations**

Top Documents WITHOUT LSI:

```
artc41.txt (score: 0.4161)
artc42.txt (score: 0.4015)
artc3.txt (score: 0.3787)
artc69.txt (score: 0.3636)
artc43.txt (score: 0.3580)
artc46.txt (score: 0.3324)
artc45.txt (score: 0.3227)
artc67.txt (score: 0.2898)
artc66.txt (score: 0.2697)
artc5.txt (score: 0.2020)
```

=====

Top Documents WITH LSI:

```
artc41.txt (score: 0.6761)
artc42.txt (score: 0.6471)
artc3.txt (score: 0.6196)
artc69.txt (score: 0.5922)
artc43.txt (score: 0.5772)
artc46.txt (score: 0.5265)
artc45.txt (score: 0.5187)
artc67.txt (score: 0.4677)
artc66.txt (score: 0.4346)
artc5.txt (score: 0.3761)
```

In this output it is seen that even though same documents were retrieved- probably due to having a smaller dataset with only 100-150 news articles, but by using LSI, higher scores were observed indicating stronger semantic alignment between the query and the retrieved documents. This suggests that LSI enhances the system's confidence in the retrieved results as the highest-scoring articles have indeed high relevance, including artc41.txt titled "Russia and Ukraine Agree Naval Ceasefire in Black Sea" and artc42.txt titled "Trump Envoy Steve Witkoff Dismisses Starmer Plan for Ukraine".

Next query: **Protests in Istanbul**

```
=====
Top Documents WITHOUT LSI:
artc22.txt (score: 0.2922)
artc19.txt (score: 0.2876)
artc4.txt (score: 0.2224)
artc21.txt (score: 0.2157)
artc87.txt (score: 0.1887)
artc20.txt (score: 0.1732)
artc18.txt (score: 0.1564)
artc92.txt (score: 0.1507)
artc23.txt (score: 0.1441)
artc73.txt (score: 0.0799)
=====
Top Documents WITH LSI:
artc22.txt (score: 0.7037)
artc19.txt (score: 0.6928)
artc4.txt (score: 0.5369)
artc21.txt (score: 0.5207)
artc87.txt (score: 0.4553)
artc20.txt (score: 0.4179)
artc18.txt (score: 0.3769)
artc92.txt (score: 0.3627)
artc23.txt (score: 0.3467)
artc73.txt (score: 0.1927)
```

Again for the comparison of cosine similarity retrieval with or without LSI for the query "Protests in Istanbul", same top documents were retrieved and ranked consistently, but with noticeably higher similarity scores in the LSI-enhanced version. For example, artc22.txt, artc19.txt, and artc4.txt were the top three results, with their scores increasing from 0.2922, 0.2876, and 0.2224 (without LSI) to 0.7037, 0.6928, and 0.5369 respectively when LSI was applied. These articles were indeed highly relevant: artc22.txt is titled "Turkey protests: Why thousands are protesting Imamoglu's arrest", artc19.txt is "Turkey protests: Sixth night of demonstrations as Erdogan hits out at unrest", and artc4.txt is "Thousands turn out in

Turkey for protests after more than 1,400 arrests”. This reinforces the effectiveness of LSI in enhancing retrieval confidence as increased similarity scores illustrate a stronger semantic match between the query and documents.

In addition to comparing top-ranked documents, a similarity threshold can also be applied to cosine scores to filter out weakly related documents. This approach ensures that only documents with a minimum level of relevance to the query are retrieved, rather than selecting a fixed number of top results. Using a threshold makes the retrieval system more realistic which can be especially useful in larger datasets.

Entered query with this approach: **South Korea president**

```
=====
Documents WITHOUT LSI (cosine similarity > 0.1):
artc10.txt (score: 0.2226)
artc103.txt (score: 0.1473)
artc2.txt (score: 0.1437)
artc1.txt (score: 0.1238)
=====
Documents WITH LSI (cosine similarity > 0.1):
artc10.txt (score: 0.4094)
artc103.txt (score: 0.2693)
artc2.txt (score: 0.2638)
artc1.txt (score: 0.2269)
artc40.txt (score: 0.1278)
artc99.txt (score: 0.1252)
artc89.txt (score: 0.1052)
```

In this example, a threshold of 0.1 was applied to filter weak matches. Without LSI, four documents were retrieved, while LSI returned seven. The top results remained consistent, but similarity scores increased noticeably—for instance, artc10.txt rose from 0.2226 to 0.4094. Additionally, LSI enabled the retrieval of extra documents such as artc40.txt, artc99.txt, and artc89.txt, which were below the threshold in the non-LSI case. This highlights how LSI not only boosts similarity scores for relevant content but also broadens the scope of retrieval by capturing deeper semantic relationships.

## 4. Discussion and Analysis

Through the experiments conducted, Latent Semantic Indexing (LSI) demonstrated consistent improvements in the retrieval system's ability to represent the semantic relevance of documents. In every test, LSI increased the similarity scores of top-ranked results, providing greater confidence in their relevance to the user's query, which increases the user's interaction with the system by better reflecting the strength of the match. When a similarity threshold was introduced, LSI proved especially effective by retrieving more documents above the threshold than traditional cosine similarity demonstrating its ability to catch documents that may not share exact keywords but are still conceptually aligned. For instance, in the "**South Korea president**" example, LSI retrieved additional relevant documents that standard cosine filtering would have excluded. This shows that LSI not only reinforces top matches but also broadens the retrieval scope in a meaningful and controlled way.

Parameter selection, particularly the number of dimensions ( $k = 100$ ), played a key role in shaping the latent space. Although this value was not optimized, it yielded stable results across the dataset. Smaller values of  $k$  risk underfitting by oversimplifying the semantic structure, while larger values may lead to overfitting by capturing noise rather than general patterns. However further tuning could enhance performance, especially in larger or more diverse datasets. Other limitations of this study may include the use of a relatively small dataset, the absence of formal relevance annotations, and a limited set of query types. Future improvements may include dynamic thresholding, evaluation with larger datasets or relevance feedback from user to further validate contextual impact.

## 5. Conclusion

In conclusion, this project demonstrated how Latent Semantic Indexing (LSI) enhances traditional vector space retrieval by capturing deeper semantic relationships between queries and documents. Across multiple political news articles, LSI consistently increased similarity scores and broadened retrieval coverage in a positive way, especially when using similarity thresholds. Although the dataset was limited in size and diversity, the findings showed that LSI improves both retrieval quality and user interaction. With further parameter tuning, larger datasets, and formal relevance evaluation, LSI-based systems could offer even more accurate and context-aware search experiences.

## 6. References

The news articles used in this project were sourced from the following media outlets:

*BBC News*. Available at: <https://www.bbc.com>

*CNBC News*. Available at: <https://www.cnbc.com>

The full implementation code and notebook used in this project can be accessed on GitHub:

[https://github.com/laraalkan/NLPandIR\\_proj1/blob/main/midterm\\_assignment/codes\\_for\\_midterm\\_assignment.ipynb](https://github.com/laraalkan/NLPandIR_proj1/blob/main/midterm_assignment/codes_for_midterm_assignment.ipynb)